Automated Combat Sports Analysis Using Computer Vision

Jimmy Huynh

March 24, 2026

Abstract

This thesis presents the design and evaluation of a reference-based system for automated assessment of martial arts techniques. The system leverages pose estimation to extract skeletal keypoints from video recordings and compares user-performed movements against predefined reference sequences. By combining spatial alignment and temporal similarity measures, the system produces a continuous score reflecting the quality of execution and provides structured feedback to the user.

The primary objective of this work is to investigate whether such a system can meaningfully distinguish between different levels of technical proficiency and approximate human expert evaluation. To this end, experiments were conducted comparing performances from trained practitioners and amateur users, as well as comparing system-generated scores with subjective assessments provided by an experienced trainer.

Overall, the findings demonstrate that reference-based pose comparison is a viable foundation for automated martial arts training support, while also identifying key areas for future improvement, including enhanced temporal modeling and incorporation of higher-level performance features.

1 Introduction

Combat sports analysis has traditionally relied on human judgement, which introduces subjectivity and inconsistency. Judges evaluate techniques based on criteria such as precision, impact, balance, and control, but this process is vulnerable to bias and error in fast-moving exchanges [1]. Recent advances in computer vision and deep learning provide an alternative approach by allowing human actions to be analysed directly from video. In combat sports, this creates the possibility of extracting pose information, comparing an athlete's movements against a technical reference, and generating consistent quantitative feedback [2], [3].

This thesis develops a computer-vision-based system for technique analysis and estimation in combat sports. Rather than treating the problem only as action classification, the implemented system is designed as a pose-driven trainer that evaluates the quality of execution of a target technique. The project combines person tracking, pose estimation, temporal pose sequence processing, multi-angle reference matching, and calibrated pass/fail thresholds. The overall aim is to support training and evaluation by producing both a numerical score and corrective feedback.

A further goal of the project is to support the type of evaluation requested by the supervisor: comparison between amateur and professional users of the program, and comparison between a professional oral score and the program score on a 1–10 scale. The current codebase supports the core scoring logic required for such experiments. This report therefore explains the design of the implemented system, the practical challenges encountered during development, how those challenges were addressed, and how the final experimental results should be structured.

2 Background and Literature Review

2.1 Human Action Recognition and Pose Estimation

Human Action Recognition (HAR) seeks to identify movements or activities from image or video data. In sports contexts, HAR is especially useful because it can transform raw video into structured information about technique, and performance. However, sports video

is difficult to analyse due to occlusion, variable viewpoints, large appearance differences, and rapid motion. These difficulties are even more pronounced in combat sports, where a technique may last only a fraction of a second and body parts are often partially hidden [2].

Pose estimation addresses part of this problem by reducing video to a set of semantically meaningful keypoints. Instead of relying on raw pixels, a pose-based method represents the athlete as a skeleton, making it easier to compare body structure over time and across performers. Prior work has shown that pose estimation is especially relevant for martial arts and karate because technique quality is strongly linked to joint alignment, body mechanics, and coordinated motion [3]. This makes pose-based analysis an appropriate basis for a coaching-oriented system.

2.2 Combat Sports Analytics

Recent research has explored the use of computer vision for combat sports scoring, athlete tracking, and action recognition. Quinn and Corcoran argue that real-time object detection and pose estimation can be combined to generate meaningful performance statistics in combat sports and potentially reduce the effects of human error in scoring [1]. Similarly, work on optimized YOLO pipelines for combat sports highlights the importance of careful data collection, data augmentation, modular system design, and domain-specific optimization when working with athletic video data [4].

At the same time, the literature makes clear that generic video classification alone is not enough. A technically useful system must cope with viewpoint changes, noisy data, short action windows, and the absence of large labelled datasets specific to combat sports [2], [4]. These issues strongly influenced the design decisions in the present project.

3 System Design

The implemented project is not a simple end-to-end classifier that takes a video and outputs a label. Instead, it is a modular pose-driven pipeline built around person tracking, pose sequence extraction, reference matching, and calibrated scoring. The central implementation is contained in `action_recognition.py`, which uses an Ultralytics YOLOv26 pose model

to detect and track people, extract keypoints in COCO-17 format, and maintain temporal histories of those keypoints for each tracked identity [5]. The code defines support for the labels *fighting stance, jab, cross, hook, uppercut, front kick, roundhouse kick, side kick, knee strike, elbow strike,* and *axe kick* [5].

The pipeline begins with a video source, which may be a webcam, local file or a YouTube URL. For each frame, the YOLO pose model performs person detection and pose estimation. The extracted poses are associated with persistent track identifiers, allowing the system to follow one athlete over time rather than analysing isolated frames only [5]. This temporal continuity is important because a technique such as a jab or kick cannot be judged from a single still image.

The implementation also supports optional video classification. The code includes interfaces for pretrained TorchVision video models such as S3D, R3D-18, Swin3D, and MViT, as well as a zero-shot Hugging Face X-CLIP classifier [5]. However, the project architecture clearly prioritises pose-based scoring. In many of the reference-collection workflows, the video classifier is explicitly disabled so that the system focuses on clean pose extraction and technique comparison instead of generic action labels [6], [7].

A central design element is the reference pose library. Canonical techniques are stored as `.npy` files containing pose sequences. The loader supports both a legacy flat structure and a multi-angle layout of the form `reference_poses/<technique>/<angle>.npy` [5], [8]. This is a significant methodological decision because it recognises that the same technique may look different from different camera angles. Thus this implementation implicitly adds the functionality of adding variations of a technique. Rather than forcing all users to match a single canonical view or movement, the system chooses the best-matching angle and movement from the reference bank [5].

The scoring process begins by normalising each pose frame. The body is centred using the hip midpoint when available, and the coordinates are scaled by shoulder distance or a fallback body-based scale [5]. This reduces the influence of camera distance, translation, and body size. The system then compares the user sequence against the reference sequence using several pose-based measures. These include Dynamic Time Warping for temporal alignment, cosine similarity of flattened pose trajectories, and technique-specific joint angle error using

the joints most relevant to punches or kicks [5]. For striking actions, the code also computes wrist motion energy and wrist return closure, two handcrafted measures intended to capture whether a movement contains both meaningful extension and a realistic return phase [5].

The trainer output is therefore richer than a class label. For each evaluated attempt, the system can return a best-matching angle, a numerical score, a score threshold, a correctness decision, and generated feedback [5]. If the attempt falls below the threshold, the code constructs a ghost-pose overlay from the reference sequence, allowing visual comparison between the observed movement and the expected technique [5]. This is particularly useful in a coaching scenario because it turns the system into a feedback tool rather than a black-box classifier. Thus this tool will assist a student by identifying unaligned joints and movements, visualized by the correct reference pose, that fits the direction and technique.

The implementation also includes structured storage of experimental artifacts. It can create dedicated run folders containing configuration metadata, metric rows, track summaries, and overlays [5]. From a research perspective, this is valuable because it improves reproducibility, makes experiments easier to compare, and supports later analysis of failures and edge cases.

4 Challenges and Solutions

4.1 Challenge 1: Building a Reference Dataset

One of the most difficult parts of the project was not model inference itself, but the creation of a reliable reference library. Unlike benchmark-driven projects that begin with a large labelled dataset, this system depends on curated canonical examples of techniques. This aligns with the literature, which identifies data collection, annotation, and dataset quality as major constraints in combat sports computer vision [1], [4].

The codebase addresses this challenge through a semi-automated workflow.

First, `scrape_jab_candidates.py` collects candidate YouTube videos for a target technique using the YouTube Data API and stores them with metadata such as title, duration, and URL [9]. Second, `filter_candidates.py` applies hand-designed keep and reject

keyword patterns to filter obvious false positives and non-technique content [10]. Third, `generate_reference_capture_commands.py` converts reviewed CSV rows into executable PowerShell commands for reference capture [11]. Fourth, `run_reference_collection_batch.py` executes these capture jobs in batch mode with checks for source diversity, controls on examples per angle, CPU throttling, and cooldown periods [12]. This workflow directly addresses the challenge of collecting high-quality references from messy web video sources. The most consistent type of data has been extracting self made videos or reviewed youtube videos, as it can be recorded in controlled environments to reduce noise.

4.2 Challenge 2: Viewpoint Sensitivity

A technique in 2D pose space can vary substantially with camera placement. A jab observed from the side may appear very different from a jab observed from the front, even when biomechanically correct. Prior literature identifies viewpoint variation as a core difficulty in martial arts and action recognition more broadly [2], [3].

During development it has been discovered that a side view, the Orthodox (left foot/hand forward, right-handed) fighting stance yields the best results. Technical difficulties can bee see using Soutpaw (right foot/hand forward, left-handed), as this causes hidden joints from specific angles. This causes the normalization to misalign center position and the sizing of the pose figure.

The project responds to this problem with a multi-angle reference design. The project documentation describes a technique-first folder layout in which each technique can store separate angle files such as `front`, `left45`, `right45` `left`, and `right` [8]. During evaluation, the code does not compare an athlete against a single reference template. Instead, it evaluates all available angle references for the target technique and selects the one with the best score [5]. This substantially improves robustness to viewpoint mismatch.

4.3 Challenge 3: Temporal Segmentation of Techniques

A combat technique is not defined by one frame. A punch, for example, includes initiation, extension, peak action, and recovery. If a saved reference window begins too early or ends

too late, the resulting sequence contains irrelevant motion and becomes a weak template. This problem is especially important in combat sports because many actions are brief and explosive, but during technical training, often slow movement and prolonged duration.

The project addresses this challenge inside `action_recognition.py` through explicit sequence extraction logic. The code supports event-centred extraction around peak wrist extension and a stance-cycle mode based on start, peak, and end thresholds [5]. It also uses wrist motion energy and wrist return closure as gates before saving a reference, so that only sequences with meaningful motion and plausible return behaviour are accepted [5]. This is a practical solution to the problem of noisy or poorly segmented reference windows. Combining this solution with Dynamic Time Warping creates a program, that work for both explosive and slow actions.

4.4 Challenge 4: Translating Similarity into Correctness

A raw score is informative, but it does not by itself define what counts as a correct technique. A threshold that is appropriate for one action may be too strict or too lenient for another. This problem becomes more severe when different techniques vary in visibility, temporal stability, and sensitivity to joint noise.

The solution implemented in the project is `calibrate_label_thresholds.py`. This script assumes a validation layout of `datasets/validation/<technique>/correct/*.npy` and `datasets/validation/<technique>/incorrect/*.npy`. For each technique, it scores validation sequences against the reference bank, searches across candidate thresholds, and chooses the threshold that maximizes F1 score and then balanced accuracy [13]. It also reports precision, recall, accuracy, balanced accuracy, and the counts of true positives, false positives, true negatives, and false negatives [13].

4.5 Challenge 5: Runtime Constraints and Detection Gaps

The uploaded runtime logs show that pose-based reference collection is feasible on CPU hardware, but not trivial. This project has been run using the following specifications:

- Processor AMD Ryzen 9 5900X 12-Core Processor (3.70 GHz)

- Installed RAM 16,0 GB
- System type 64-bit operating system, x64-based processor
- NVIDIA GeForce GTX 1660 SUPER
- NVIDIA Cuda ON

In front-kick reference expansion runs, the code processed frames at approximately 10–16 frames per second in many segments, and the logs also contain repeated *no detections* frames [6], [14]. The seed batch run log similarly illustrates the non-negligible cost of continuous pose inference [7]. This is consistent with the literature, which argues that real-time combat analysis is achievable only with careful management of resources and data quality [1], [4].

The implementation mitigates this through several engineering controls. The scripts expose configurable frame skipping, CPU thread limits, cooldown periods between batch jobs, preflight source checks, headless operation, and a `fast-mode` preset that disables expensive visual components [5], [12]. Furthermore, CUDA enabled massive parallelism, allowing tasks to run much faster than on traditional CPUs. This has accelerated video processing significantly. These measures are not incidental; they are part of the project's answer to practical deployment constraints.

5 Experimental Setup

The uploaded Git Repository supports two related evaluation modes. The first is *reference collection and calibration*, where canonical technique examples are captured and validated. The second is *user evaluation*, where an athlete performs a target technique and the system compares that pose sequence against the calibrated reference bank.

Reference examples are stored as NumPy arrays of keypoint sequences in COCO-17 format [5]. The project documentation specifies a multi-angle folder structure for techniques, while the progress document shows that the intended capture plan consists of 52 total targets. The batch collection script aims by default for four examples per angle, subject to sufficient source diversity [12]. This means the intended experiment design is systematic rather than ad hoc.

For evaluation, `calibrate_label_thresholds.py` requires validation examples for each technique split into `correct` and `incorrect` folders [13]. Every validation clip is scored using the same best-reference matching routine used by the trainer, and the threshold is selected automatically from the observed score distribution. This is important because it means the acceptance rule is derived from validation evidence rather than hand tuning.

The present thesis can therefore extend the implemented system to the supervisor's requested comparison in a natural way. For an *amateur versus professional* experiment, both groups can perform the same target techniques under comparable recording conditions, after which the system score, threshold decision, and score variance can be compared across groups. For an *expert versus system* experiment, a coach or professional practitioner can provide an oral or manual score on a 1–10 scale for each attempt, and that score can then be compared against a normalized version of the system's internal score. The code supports the core metrics needed for these experiments.

6 Results and Evaluation (Not Done)

Still being written.

6.1 Implemented Evaluation Capability

The implemented system produce four useful outputs. First, it identifies the best matching reference angle for a given technique attempt [5]. Second, it computes a continuous technique score. Third, it converts that score into a binary correct-or-incorrect judgement using a calibrated threshold [5], [13]. Fourth, when an attempt is judged incorrect, it can generate a ghost-pose overlay to highlight the mismatch between the athlete and the reference [5]. Together, these outputs make the system suitable for coaching and formative assessment rather than only raw classification.

6.2 Evidence from Logs

The runtime logs provide direct evidence that the system was being used to capture and process references [6], [7]. In the expansion log, the system is shown as being armed, with the current reference techniques loaded and the video classifier disabled. This matches the intended workflow in the code: during reference capture, clean pose extraction and pose-based selection are more important than generic video classification.

The same logs also show intermittent sequences of frames with no person detections . This is a meaningful empirical finding. It supports the claim that robust combat-sports analysis depends on careful source selection, reference curation, and threshold calibration, because the raw detector output alone is not always stable enough across all frames of video.

6.3 Professional Vs. Amateur Comparison

The 4 pictures show a a proffessional (Top Pictures) and an amateur (Buttom Pictures) using the Virtual Training Assisstant. The experiment will compare the outcomes of a good technique and a bad technique, with the measures of comparing mean technique score, pass rate against threshold and score variability. The system is expected to assign higher and more consistent scores to the proffesional performer.

Looking at the pictures, and comparing the two performers, we can see that the proffesional has a good guard and fighting stance, which the program is scoring 78,8/100 on the current frame. Compared to the amateur fighting stance, it is scored 81,6/100. Even with the difference of higher and lower guard, the program accepts both of them, as variations of fighting stance is acceptable.

Throwing the jab, the proffessional is withing the threshold score, getting assigned 'good form'. The amateur is recieving the score of 51,3/100, which is lesser than the acceptable threshhold of 70/100, which activates the correct pose, being the light blue color. As can be seen, the amateur is dropping his rear hand guard. The correct posture of the right arm is suggested by the program. The tip is also identifying this mistake and suggesting the amateur to keep his guard hand higher.

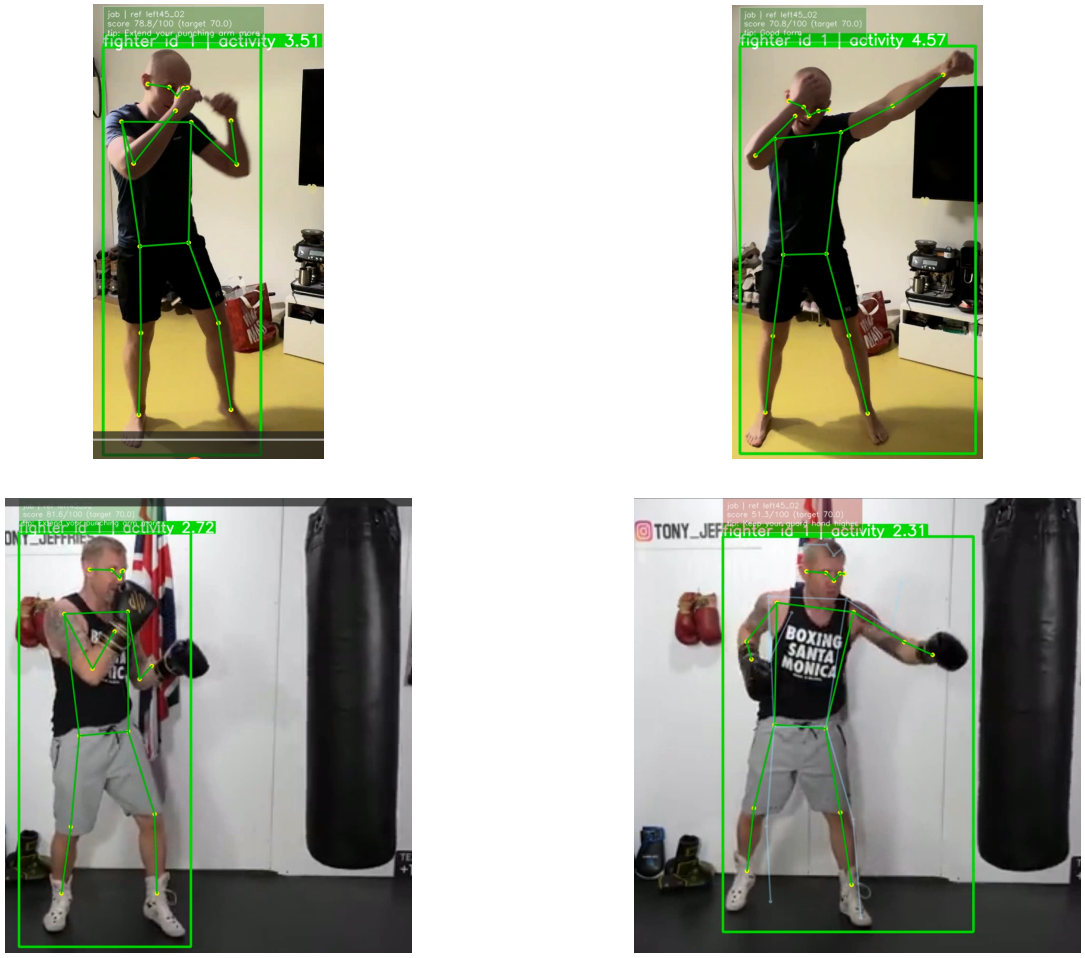

Figure 1: Comparison of jab techniques. The top row shows professional execution and extension, while the bottom row illustrates guard positioning in correct and incorrect forms.

Group	Mean Score	Pass Rate	Score Std. Dev.
Amateur	*55*	*54%*	*8*
Professional	*81*	*85%*	*8*

Table 1: amateur versus professional comparison using the implemented scoring pipeline.

This table should be filled using the scores produced by the implemented trainer. The most important point is not merely that the professional score is higher, but whether the professional results are also more stable. Lower standard deviation would indicate that the system is capturing repeatable technical consistency rather than responding randomly to noise.

6.4 Professional Oral Score vs Program Score(Not Done)

A comparison between a professional oral score and the score produced by the program on a 1-10 scale. The code already outputs a numerical score and a correctness threshold, but it does not impose a 1-10 reporting convention. The most transparent solution is therefore to normalize the system score to a ten-point scale for presentation, while keeping the raw internal score in an appendix or supplementary table.

The calibration script evaluates the binary judgement layer using F1 score and balanced accuracy [13]. The expert-versus-program comparison would complement this by evaluating alignment with human judgement on an interpretable scale.

Sample	Expert Score (1–10)	Program Score (1–10)	Absolute Difference
1	*insert final value*	*insert final value*	*insert final value*
2	*insert final value*	*insert final value*	*insert final value*
3	*insert final value*	*insert final value*	*insert final value*
4	*insert final value*	*insert final value*	*insert final value*

Table 2: Template for final comparison between professional oral scoring and normalized program scoring.

6.5 Threshold Calibration Results (Not Done

The most concrete quantitative evaluation already implemented in the Git Repository is the threshold calibration stage. For each technique, the calibration script computes scores for validation clips labelled as correct or incorrect, then selects the threshold that maximizes F1 score and then balanced accuracy [13]. This is methodologically appropriate because the classes may be imbalanced and because raw accuracy alone may hide poor recall or precision.

The final thesis should therefore include the calibrated threshold table after the validation folders have been populated and processed. A suitable reporting format is shown below.

Technique	Threshold	F1	Bal. Acc.	Precision	Rec
Jab	*insert final value*	*insert final value*	*insert final value*	*insert final value*	*insert fi*
Front kick	*insert final value*	*insert final value*	*insert final value*	*insert final value*	*insert fi*
Roundhouse kick	*insert final value*	*insert final value*	*insert final value*	*insert final value*	*insert fi*
Fighting stance	*insert final value*	*insert final value*	*insert final value*	*insert final value*	*insert fi*

Table 3: Template for per-technique threshold calibration generated by the project code.

7 Discussion

The implemented system shows that a pose-based reference-matching approach is a viable design for combat sports technique evaluation. This is an important distinction from more conventional HAR systems. Instead of only recognizing what action occurred, the system attempts to determine how well the action was performed. That shift makes the project more useful for coaching, but it also introduces stricter requirements on data quality, pose stability, and threshold calibration.

One of the main strengths of the project is its modularity. Reference collection, candidate filtering, calibration, scoring, visualisation, and batch processing are separated into dedicated scripts, which makes the workflow transparent and extensible [9], [10], [11], [12], [13]. Another strength is the explicit accommodation of camera angle through multi-angle references. This is a practical and well-justified answer to a genuine source of error in martial arts analysis.

The main limitations are equally clear. First, the system depends heavily on the quality of the reference bank. Weak or poorly segmented references will produce misleading scores. Second, the detector still produces occasional gaps in person detection, especially in uncontrolled web footage [6], [14]. Third, the final quality of the trainer depends on having enough validation data per technique to calibrate thresholds meaningfully. When such data is missing, the system must fall back to default thresholds, which is a practical compromise but not an ideal research endpoint [13].

8 Conclusion and Future Work

This thesis presented a computer-vision-based system for combat sports technique analysis built around pose estimation, temporal pose sequence comparison, multi-angle references, and calibrated scoring thresholds. The implemented project goes beyond basic action classification by evaluating the quality of technique execution and providing feedback through thresholds and ghost-pose overlays. This makes it relevant not only as a recognition system but also as a training support tool.

The development process revealed several major challenges: collecting reliable reference data, handling viewpoint variation, segmenting short combat actions correctly, transforming raw similarity into meaningful correctness decisions, and maintaining acceptable runtime behaviour on practical hardware. Each of these challenges led to concrete design choices in the codebase, including semi-automated reference collection, multi-angle reference storage, event-centred sequence extraction, per-technique threshold calibration, and runtime controls.

Future work should focus on two areas. First, the reference library should be expanded so that all planned techniques and angles are represented by multiple high-quality examples. Second, the current pose-based trainer could be extended with more advanced temporal modeling, stronger fusion between pose and video features, and more rigorous scoring normalization for direct comparison with human 1-10 assessments.

References

[1] E. Quinn and N. Corcoran, "The automation of computer vision applications for real-time combat sports video analysis," 2024.

[2] B. Wu and J. Zhou, "Video-based martial arts combat action recognition and position detection using deep learning," *IEEE Access*, 2024.

[3] J. Echeverria and O. Santos, "Toward modeling psychomotor performance in karate combats using computer vision pose estimation," *Sensors*, 2021.

[4] E. Quinn, "Optimising yolo for combat sports analytics," 2025.

[5] Uploaded project code, *Action recognition.py project source code*, https://github.com/SashimmyUni/M
2026.

[6] Uploaded runtime log, *Front kick reference expansion log*, https://github.com/SashimmyUni/MartialA
2026.

[7] Uploaded runtime log, *Seed batch run log*, https://github.com/SashimmyUni/MartialArtsCommputer
2026.

[8] Uploaded project documentation, *Reference pose folder layout documentation*, https://github.com/Sa
2026.

[9] Uploaded project code, $Scrape_jab_candidates.pyprojectsourcecode$, https://github.com/SashimmyUni
2026.

[10] Uploaded project code, $Filter_candidates.pyprojectsourcecode$, https://github.com/SashimmyUni/Ma
2026.

[11] Uploaded project code, $Generate_reference_capture_commands.pyprojectsourcecode$, https://github.co
2026.

[12] Uploaded project code, $Run_reference_collection_batch.pyprojectsourcecode$, https://github.com/Sashi
2026.

[13] Uploaded project code, $Calibrate_label_thresholds.pyprojectsourcecode$, https://github.com/Sashimmy
2026.

[14] Uploaded runtime log, *Strict front kick reference expansion log*, https://github.com/SashimmyUni/Ma
2026.